Villanova LCNC Component

M3 – Multilingual Speech Synthesis

Overview

The M3 – Multilingual Speech Synthesis module is part of the predefined component catalog available within the Villanova LCNC Platform. It provides a fully offline text-to-speech (TTS) pipeline based on Argos Translate for multilingual translation, Chatterbox TTS as the default neural speech engine, and Piper TTS as an alternative ONNX-based engine.

The Villanova Platform is an open-source, low-code/no-code framework that enables users to build, customize, and deploy AI-driven vertical applications. This module can be integrated into LCNC workflows to generate speech in five languages (Italian, English, Spanish, French, German) starting from a single text input.

Key Features

1) Multilingual Text Input & Automatic Language Detection
   The component accepts plain text input in any of the supported languages (it, en, es, fr, de). The source language is automatically detected so that users do not need to manually specify it.

   2) Offline Translation (Argos Translate)
   The module uses Argos Translate to translate the input text into all target languages. All translation models run fully offline once the Argos packages have been installed. The full translation matrix among the five supported languages (up to 20 language pairs) can be used, and the system gracefully skips languages for which models are missing.

   3) Default Neural TTS with Chatterbox
   Chatterbox Multilingual TTS is the default TTS engine. It uses a single multilingual model capable of synthesizing speech for all supported languages and can leverage GPU acceleration (CUDA) when available. If Chatterbox is not installed or fails at runtime, the component transparently falls back to Piper.

   4) Alternative ONNX TTS with Piper
   Piper TTS acts as a lightweight alternative engine. It provides fast, fully

offline synthesis using per-language ONNX voice models. Piper can be explicitly selected from the CLI (for example with the --piper or --engine piper options, depending on the script) when a smaller runtime footprint or deterministic behavior is preferred.

5) Multi-Output Speech Synthesis
For each input text, the component translates and synthesizes speech for all configured target languages. The standard configuration generates up to five WAV files, one for each of it/en/es/fr/de, in a single run.

6) Robust Processing Pipeline
The pipeline includes logging of translation and synthesis steps, detection of missing models, and partial-output tolerance. When a specific language fails (for example due to missing models), the component continues for the remaining languages.

7) Optional Streamlit UI
The repository includes a Streamlit-based UI application that provides an interactive interface on top of the same multilingual pipeline implemented in tts_multi_language.py. Users can type or upload text, choose the engine, trigger multilingual synthesis, listen to the audio, and download the generated WAV files.

8) CLI Interfaces
The module provides two main CLI utilities for production-oriented use: *tts_mono_language.py* for single-language generation and *tts_multi_language.py* for multilingual generation. Additional test scripts are available to exercise each engine individually.

Known Limitations

• Officially tested and recommended on Ubuntu 22.04+.
• Piper voice models (.onnx and metadata files) must be downloaded and placed in the expected directory.
• The Chatterbox multilingual checkpoint is large (~1.8 GB) and must be downloaded at least once.
• Argos Translate requires the appropriate language-pair packages to be installed for offline translation.
• Only the five predefined languages (it, en, es, fr, de) are supported out of the box.
• No built-in retry or rate limiting, as the component is designed for local offline execution.

Supported LCNC Platform Versions

• Compatible with Villanova LCNC Platform ≥ 1.0.

Requirements

Python Version

• Python 3.11+ (recommended for best compatibility with Chatterbox and audio libraries).

Python Libraries

• argostranslate — offline translation.
• chatterbox-tts — default multilingual neural TTS engine.
• piper-tts — alternative ONNX-based TTS engine.
• langdetect — automatic language detection.
• numpy, soundfile, torch — audio and model runtime dependencies.
• streamlit — required only for the optional UI application.

Models

• Piper models for Italian, English, Spanish, French, German (.onnx + JSON metadata).
• Argos Translate language-pair packages for the desired translation matrix.
• Chatterbox multilingual checkpoint (downloaded at first use).

Example of Operation

Use Case

A user provides text in any of the supported languages. The system detects the source language, translates the text into the configured target languages, and generates one WAV file per language using Chatterbox by default, with Piper as a fallback or alternative engine.

Programmatic Invocation (Multilingual Pipeline)

The core multilingual pipeline is implemented in tts_multi_language.py via the offline_multi_tts function. It detects the source language, translates into the full it/en/es/fr/de set, and synthesizes the corresponding WAV files:

```
from tts_multi_language import offline_multi_tts

text = "Good morning, it is a pleasure to meet you."
offline_multi_tts(text, use_chatterbox=True)
```

By default, use_chatterbox=True selects the Chatterbox engine when available;

setting use_chatterbox=False forces Piper instead. The function writes WAV files under chatterbox_multi_output/ or piper_multi_output/.

CLI Invocation – Single-Language TTS (tts_mono_language.py)

The tts_mono_language.py script generates a single WAV file from a text string or a .txt file. It automatically detects the input language and, if no target language is specified, synthesizes in the same language. If a different target language is provided, it attempts translation with Argos Translate before synthesis.

Key CLI parameters:
--text "..." Direct text input (mutually exclusive with --file).
--file path.txt UTF-8 .txt file input (mutually exclusive with --text).
--lang xx Target language (it, en, es, fr, de). If omitted, uses the detected language.
--engine chatterbox|piper Forces the TTS engine. If omitted, the script uses Chatterbox when available, otherwise Piper.
--max-chars N Maximum chunk length for long texts.

Examples:
1) Synthesis in the same language (default engine: Chatterbox if available):
python3 ttts_mono_language.py --text "This is a test."

2) Translation + synthesis in English:
python3 ttts_mono_language.py --text "Questo è un test." --lang en

3) From file, forcing Piper as engine:
python3 ttts_mono_language.py --file input.txt --engine piper

CLI Invocation – Multilingual TTS (tts_multi_language.py)

The tts_multi_language.py script is the main multilingual CLI entry point. It detects the language of the input text, translates it into the full it/en/es/fr/de set, and synthesizes up to five WAV files in a single run. The CLI is thin: the only optional flags are used to select the TTS engine, and the remaining argument is interpreted as either a text string or a path to a .txt file.

Engine selection flags:

- --chatterbox Forces the use of Chatterbox as the engine.
- --piper Forces the use of Piper as the engine.

If neither flag is provided, Chatterbox is used by default when installed; otherwise

Piper is used as a fallback.

Examples:
1) Multilingual synthesis from a literal text string (default engine: Chatterbox if available):
```
python3 tts_multi_language.py "This is a multilingual TTS test."
```

2) Multilingual synthesis from a .txt file, forcing Chatterbox (explicit):
```
python3 tts_multi_language.py --chatterbox input.txt
```

3) Multilingual synthesis from a .txt file, forcing Piper as engine:
```
python3 tts_multi_language.py --piper input.txt
```

Outputs are written under chatterbox_multi_output/ when Chatterbox is used and under piper_multi_output/ when Piper is used.

CLI Invocation – Engine Test Scripts

Two additional test scripts are included for engine-level diagnostics and quick checks:

• Piper TTS test:
```
python3 piper_audio_test.py --text "Hello world" --lang it
```

• Chatterbox TTS test:
```
python3 chatterbox_audio_test.py --text "Bonjour" --lang en
```

These scripts are primarily intended for testing and are less important than ttts_mono_language.py and tts_multi_language.py for production workflows.

Streamlit UI Invocation

The Streamlit UI is built on top of the same multilingual pipeline defined in tts_multi_language.py. From the project root, it can be launched with:

```
streamlit run app.py
```

The app opens a local web interface (by default on http://localhost:8501) where users can enter or upload text, select the engine, and trigger multilingual synthesis. Generated files are stored in the same output directories used by the CLI scripts.

Advanced Configuration

Customizable Runtime Parameters

• Engine selection (Chatterbox vs Piper) at CLI level or programmatically.
• Model directories for Piper voices and Argos translation packages.
• Maximum chunk length and pause duration for long-text synthesis.
• Device selection for Chatterbox (CPU vs CUDA GPU).

Pipeline Customization

• Extension or restriction of the set of supported languages by adding or removing models.
• Integration of logging with LCNC-wide logging and monitoring solutions.
• Custom fallback strategies between Chatterbox and Piper depending on quality or performance requirements.

Integration with other LCNC Components

The M3 component can be integrated in multiple stages of Villanova LCNC workflows:

Upstream Components

• Text ingestion modules.
• Chatbots and conversational agents.
• Summarization and content-transformation pipelines (e.g. document → summary → speech).

Core Workflow Role

• Multilingual speech generation from user input, system messages, or documents.
• Voice-over for automated training content, tutorials, and notifications.
• Accessibility features for reading content aloud in multiple languages.

Downstream Components

• Audio players in LCNC front-end applications.
• Multimedia automation pipelines combining text, audio, and video.
• Storage, indexing, and delivery of generated audio assets.

Repository Structure

/app.py	– Streamlit UI for interactive use
/tts_multi_language.py	– core multilingual TTS pipeline and CLI
/ttts_mono_language.py	– single-language TTS pipeline and CLI
/piper_audio_test.py	– Piper engine test CLI
/chatterbox_audio_test.py	– Chatterbox engine test CLI
/argos_pkgs/	– optional directory for offline Argos packages
/requirements.txt	– Python dependencies
/README.md	– module overview and setup instructions
/LICENSE	– project license

License

This component is released under the Apache 2.0 license, unless otherwise specified by Villanova.ai SpA.

Support / Contact

For assistance, issues, or feature requests:

- Website: https://www.villanova.ai
- Email: support@villanova.ai